

GeoFS Browser Exploration — Compact Working Notes

Goal

Map safe browser-exposed GeoFS surfaces for later custom telemetry, controls, aircraft/model insertion, and wake/visual experiments.

Exploration rule:

Inspect first. Prefer read-only probes. Write only to harmless objects like labels/models until behavior is understood.

Verified roots

```
window.geofs
window.geofs.aircraft.instance
window.geofs.animation.values
window.controls
window.geofs.autopilot
window.geofs.api
window.geofs.frameCallbackStack
```

GeoFS coordinate convention

GeoFS LLA order:

```
[latitude_deg, longitude_deg, altitude_m]
```

Cesium internally uses:

```
longitude, latitude, altitude
```

Verified from API source:

```
Cesium.Cartesian3.fromDegrees(11a[1], 11a[0], 11a[2])
```

Frame callback API

```
const id = geofs.api.addFrameCallback(callback, namespace, maxExecutionTime);
geofs.api.removeFrameCallback(id, namespace);
```

Default namespace:

```
"global"
```

Callback receives:

```
geofs.api.precisionTime
```

Verified current callback rate:

```
~63.47 Hz
```

Use this for telemetry sampling.

Labels / low-risk world placement

Verified:

```
const label = geofs.api.addLabel("DOC_PROBE", lla, { font: "14px sans-serif" });
geofs.api.setLabelPosition(label, lla);
geofs.api.updateLabelText(label, text);
geofs.api.removeLabel(label);
geofs.api.getScreenCoordFromLla(lla);
```

Negative screen ☐ means above/outside viewport, not failed coordinates.

Main telemetry roots

Root	Best use
<code>geofs.aircraft.instance</code>	raw/live aircraft state, physics, rigid body
<code>geofs.animation.values</code>	clean derived/instrument telemetry

Recommended read-only telemetry packet:

```

const a = geofs.aircraft.instance;
const rb = a.rigidBody;
const v = geofs.animation.values;

const telemetry = {
  time: geofs.api.precisionTime,

  lla: a.llalocation?.slice(),
  htr: a.htr?.slice(),
  velocityENU: a.velocity?.slice(),
  speedMS: a.velocityScalar,
  velocityDir: a.velocityDirection?.slice(),

  rbLinearVelocityENU: rb.v_linearVelocity?.slice(),
  rbAngularVelocity: rb.v_angularVelocity?.slice(),
  rbAcceleration: rb.v_acceleration?.slice(),
  rbAngularAcceleration: rb.v_angularAcceleration?.slice(),
  rbJerk: rb.v_jerk?.slice(),
  rbMass: rb.mass,

  headingDeg: v.heading360,
  tiltDeg: v.atilt,
  rollDeg: v.aroll,
  altitudeMeters: v.altitudeMeters,
  altitudeFeet: v.altitude,
  haglMeters: v.haglMeters,
  kias: v.kias,
  ktas: v.ktas,
  airspeedMS: v.airspeedms,
  groundSpeedMS: v.groundSpeed,
  groundSpeedKnt: v.groundSpeedKnt,
  verticalSpeedFPM: v.verticalSpeed,
  aoaDeg: v.aoa,
  loadFactor: v.loadFactor,

  throttle: v.throttle,
  brakes: v.brakes,
  gearPosition: v.gearPosition,
  flapsPosition: v.flapsPosition,
  stalling: v.stalling,
  crashed: a.crashed,
  groundContact: a.groundContact
};

```

Attitude mapping

Verified:

```
animation.heading360 = wrap360(aircraft.htr[0])
animation.atilt = aircraft.htr[1]
animation.aroll = aircraft.htr[2]
```

Use:

```
headingDeg = geofs.animation.values.heading360;
tiltDeg = geofs.animation.values.atilt;
rollDeg = geofs.animation.values.aroll;
```

`animation.values.pitch/roll/yaw` are control/animation-ish values, not the preferred physical Euler attitude fields.

Velocity frame convention

Verified by finite-difference LLA over ~3 seconds.

Conclusion:

```
aircraft.velocity ≈ [east_mps, north_mps, up_mps]
rb.v_linearVelocity ≈ [east_mps, north_mps, up_mps]
```

So GeoFS velocity is ENU-like:

```
x = east, y = north, z = up
```

Heading relation:

```
headingDeg = (Math.atan2(east, north) * 180 / Math.PI + 360) % 360;
```

Vertical speed relation:

```
verticalSpeedFPM ≈ velocityENU[2] * 196.850394
```

Ground speed relation:

```
groundSpeedMS ≈ sqrt(east^2 + north^2)
```

Rigid-body physics root

```
const rb = geofs.aircraft.instance.rigidBody;
```

Useful fields:

```
rb.mass  
rb.s_inverseMass  
rb.v_linearVelocity  
rb.v_angularVelocity  
rb.v_acceleration  
rb.v_angularAcceleration  
rb.v_jerk  
rb.v_totalForce  
rb.v_totalTorque  
rb.gravityForce  
rb.v_localInvInertia  
rb.m_worldInvInertiaTensor
```

Verified sample:

```
rb.mass = 300000  
rb.s_inverseMass = 0.000003333333333333333333  
rb.gravityForce = [0, 0, -2943000]  
2943000 / 300000 = 9.81
```

`htrAngularSpeed` was observed as `[0,0,0]`; use `rb.v_angularVelocity` instead.

`rb.v_totalForce` and `rb.v_totalTorque` may show zero in normal console snapshots, likely because they are cleared after integration.

Acceleration distinction

Do not assume equality:

```
geofs.aircraft.instance.rigidBody.v_acceleration !==  
geofs.animation.values.acceleration  
geofs.aircraft.instance.rigidBody.v_angularAcceleration !==  
geofs.animation.values.angularAcceleration
```

Use `rb.*` for raw dynamics. Use `animation.values.*` for instrument/display-derived telemetry.

Controls API summary

Main control object contains:

```
controls.update
controls.updateKeyboard
controls.updateMouse
controls.updateJoystick
controls.updateTouch
controls.updateOrientation
controls.setters
controls.axisSetters
controls.states
controls.keyboard
controls.mouse
controls.recenter
controls.setMode
controls.reset
controls.trimUp / trimDown
controls.animatePart
```

Current mode observed:

```
controls.mode = "mouse"
```

Discrete controls:

```
controls.setters
```

Useful setter names and behavior:

setGear	toggles controls.gear.target, blocked on ground unless
allowed/debug	
setGearUp	sets gear target up
setGearDown	sets gear target down
setFlaps	applies current controls.flaps.target
setFlapsUp	decrements flaps target
setFlapsDown	increments flaps target
cycleFlaps	cycles 0..flapsSteps
setBrakes	sets aircraft.brakesOn=true, controls.brakes=1; unset
reverses	
parkingBrake	toggles aircraft.brakesOn and controls.brakes
toggleEngines	calls aircraft.stopEngine/startEngine
toggleAutoPilot	calls geofs.autopilot.toggle()
setAirbrakes	toggles controls.airbrakes.target
setAirbrakesUp	controls.airbrakes.target=1

```
setAirbrakesDown controls.airbrakes.target=0
trimUp/Down      sets states.elevatorTrimUp/Down until unset
trimNeutral       controls.elevatorTrim=0
```

Aircraft prototype methods now verified:

```
startEngine / stopEngine
reset
place
render
change / load / loadWithLivery / loadDefault / loadLivery
crash
setVisibility
getCurrentCoordinates
placeParts / placePart
loadCockpit
addShadow / removeShadow
unloadAircraft
```

Useful direct methods:

```
geofs.aircraft.instance.startEngine()
geofs.aircraft.instance.stopEngine()
geofs.aircraft.instance.reset()
geofs.aircraft.instance.place(lla, htr)
geofs.aircraft.instance.getCurrentCoordinates()
geofs.aircraft.instance.setVisibility(bool)
```

Axis controls:

```
controls.axisSetters
```

Useful axis names and behavior:

pitch	controls.rawPitch = e * joystickSensitivity
roll	controls.roll = e * joystickSensitivity
yaw	controls.yaw = e * joystickSensitivity
throttle	controls.throttle = (e + 1) / 2
brakes	controls.brakes = e
flapsPosition	controls.flaps.positionTarget = (e + 1)/2 * maxPosition;
animate	
airbrakesPosition	controls.airbrakes.target = (e + 1)/2; animate
reverse	controls.reverse = (e + 1)/2
throttlerreverse	if reverse aircraft: controls.throttle=e else (e+1)/2

Axis input convention:

```
pitch/roll/yaw e range: usually [-1, 1]
throttle e range: [-1, 1] maps to [0, 1]
brakes e range: direct, likely [0, 1]
flaps/airbrakes e range: [-1, 1] maps to [0, max]
```

Descriptor shapes:

```
// discrete
{ label, set, unset?, overridesAutopilot?, overridesPeer? }

// axis
{ label, process, overridesAutopilot?, overridesPeer? }
```

Important:

Native controls.update owns final continuous control values each frame.
Best continuous injection is a post-update wrapper around controls.update.

Autopilot API summary

Main root:

```
geofs.autopilot
```

Observed state:

```
geofs.autopilot.on = true
geofs.autopilot.mode = "HDG"
geofs.autopilot.speedMode = "knots"
```

Useful methods:

```
geofs.autopilot.turnOn()
geofs.autopilot.turnOff()
geofs.autopilot.toggle()
geofs.autopilot.setMode(mode)
geofs.autopilot.setCourse(deg)
geofs.autopilot.setAltitude(feet)
geofs.autopilot.setSpeed(knotsOrMach)
geofs.autopilot.setSpeedMode("knots" | "mach")
```



```
geofs.autopilot.setVerticalSpeed(fpm)
geofs.autopilot.resetPIDs()
```

Autopilot values:

```
geofs.autopilot.values = {
  altitude,
  course,
  speed,
  verticalSpeed
}
```

PID names:

```
pitchAngle
elevatorPitch
bankAngle
aileronRoll
throttle
```

Aircraft-specific autopilot definition sample:

```
{
  pitchAnglePID: [0.002, 0.001, 0.01],
  elevatorPitchPID: [0.05, 0.0001, 1e-7],
  bankAnglePID: [0.3, 0.001, 0.01],
  aileronsRollPID: [0.5, 0.001, 0.01],
  targetBankAngleRatio: 1
}
```

Autopilot writes to native controls through its update loop, so custom continuous control should normally disable or bypass native autopilot first.

Aircraft method note

The own-enumerable method probe returned:

```
{}
```

So aircraft methods are likely on the prototype or non-enumerable. Use prototype probing next if needed.

Model/world API shortlist

Useful later:

```
geofs.api.loadModel(optionsOrUrl)
new geofs.api.Model(url, options)
geofs.api.addModelToWorld(model)
geofs.api.removeModelFromWorld(model)
geofs.api.destroyModel(model)
geofs.api.setModelPositionOrientationAndScale(model, lla, htr, scale)
geofs.api.setModelVisibility(model, visible)
geofs.api.setModelOpacity(model, alpha)
geofs.api.setModelColor(model, r, g, b, a)
```

Likely transform convention:

```
lla = [lat_deg, lon_deg, alt_m]
htr = [heading_deg, tilt_deg, roll_deg]
scale = scalar or [sx, sy, sz]
```

Custom-control wrapper — verified working

Final verification:

```
{
  enabled: true,
  roll: 0.15,
  pitch: 0,
  yaw: 0,
  throttle: null,
  native: {
    roll: 0.15,
    rawPitch: 0,
    pitch: 0,
    yaw: 0,
    throttle: 0.5942432422902927
  }
}
```

Debug:

```
debug.appliedCount = 26
lastAfter.roll = 0.15
lastAfter.rawPitch = 0
```

```
lastAfter.pitch = 0  
lastAfter.yaw = 0
```

Conclusion:

Post-update wrapper around controls.update works.
It can override roll/rawPitch/yaw every frame.
Throttle can be left untouched using throttle:null.

Current safe control API behavior:

```
geoControl.enable(); // disables native autopilot and enables custom  
override  
geoControl.disable(); // stops overriding native controls  
geoControl.neutral(); // roll/pitch/yaw to zero, throttle=null  
geoControl.set({ roll, pitch, yaw, throttle });
```

Important:

pitch command maps to controls.rawPitch.
controls.pitch must be recomputed as controls.rawPitch +
controls.elevatorTrim.
throttle:null means do not override throttle.
throttle:0 means command idle throttle.

Next implementation target

Convert this into a Tampermonkey structure with:

1. telemetry read API
2. discrete command API
3. continuous command API
4. Y-key enable/disable toggle
5. clean restore/uninstall function